

CSE-214 Online

Behavioural Design Pattern

Duration: 25 Minutes

Problem: Online Exam Evaluation Process

An online examination platform supports multiple types of exams. Although the details of evaluating each exam type are different, the overall evaluation process follows the same general sequence.

```
Validate Submission
->
Evaluate Answers
->
Calculate Score
->
Apply Adjustment
->
Publish Result
```

The overall sequence must remain the same for all exam types. However, some individual steps may behave differently depending on the type of exam.

For this problem, consider the following three exam types:

- MCQ Exam
- Programming Exam
- Written Exam

Common Evaluation Process

1. Validate Submission

The system first checks whether the submission is valid. If validation fails, the evaluation process stops immediately and the result must not be published. If validation succeeds, the system continues to evaluate the submission.

2. Evaluate Answers

The answers are evaluated according to the rules of the corresponding exam type. The evaluation process may produce a raw score as well as other information required by later stages.

3. Calculate Score

After evaluation, the system calculates the initial score of the student. The method of calculating the score may differ for different types of exams.

4. Apply Adjustment

After the initial score is calculated, the system may apply an additional penalty or bonus. Some exam types may require an adjustment, while others may not.

5. Publish Result

If all previous steps complete successfully, the final score is published.

```
0 <= Final Score <= 100
```

Exam-Specific Rules

MCQ Exam

An MCQ exam consists of automatically evaluated questions. During evaluation:

- Each correct answer earns marks.
- Each incorrect answer may receive negative marking.
- Unanswered questions receive no marks.

For this problem, assume:

```
Correct Answer  -> +1 mark
Wrong Answer   -> -0.25 mark
Unanswered     -> 0 mark
```

After calculating the score, no additional bonus is applied. The final score cannot be negative.

Example:

```
Correct Answers : 70
Wrong Answers   : 20
Unanswered      : 10
```

The initial score is:

```
70 - (20 x 0.25) = 65
```

Programming Exam

A Programming Exam submission contains source code. The system first validates the submission by checking whether the submitted program can be compiled successfully.

If compilation fails:

```
Validation Failed
Result is not published.
```

If compilation succeeds, the program is executed against a set of automated test cases. Assume the raw score is determined from the percentage of passed test cases.

For example:

```
Total Test Cases : 20
Passed Test Cases : 16

Raw Score = 80
```

Plagiarism Penalty

After the raw score is calculated, the system checks whether plagiarism has been detected. If plagiarism is detected, a 50% penalty is applied to the calculated score.

Example:

```
Raw Score      = 80
Plagiarism Detected

Final Score = 40
```

If plagiarism is not detected, no penalty is applied.

Written Exam

For a Written Exam, the answer script is manually evaluated and the obtained marks are provided to the system.

For example:

```
Evaluated Score = 72
```

The system may optionally apply a moderation bonus. If moderation is enabled:

```
Moderation Bonus = +5 marks
```

For example:

```
Evaluated Score = 72
Moderation      = Enabled

Final Score = 77
```

The final score must not exceed 100. If moderation is disabled, the score remains unchanged.

Important Behaviour

The overall evaluation process must always follow the same sequence:

```
Validate
->
Evaluate
->
Calculate
->
Adjust
->
Publish
```

Individual exam types may provide their own behaviour for particular stages. For example:

```
MCQ Exam
  Evaluation -> Automatic answer checking
  Adjustment -> Negative marking already considered

Programming Exam
  Validation -> Compilation check
  Evaluation -> Automated test cases
  Adjustment -> Possible plagiarism penalty

Written Exam
  Evaluation -> Manually supplied score
  Adjustment -> Optional moderation bonus
```

Some stages may effectively perform no additional work for a particular exam type, but the overall evaluation procedure must remain unchanged.

Example Scenarios

Scenario 1: MCQ Exam

```
Submission validated.

Correct Answers : 70
Wrong Answers   : 20

Calculated Score = 65
Final Score      = 65

Result Published.
```

Scenario 2: Programming Exam

```
Compilation successful.

Passed Test Cases : 16 / 20
```

Calculated Score = 80

Plagiarism detected.
Penalty applied.

Final Score = 40

Result Published.

Scenario 3: Programming Exam with Compilation Error

Compilation failed.

Evaluation stopped.
Result not published.

Scenario 4: Written Exam

Submission validated.

Evaluated Score = 98
Moderation = Enabled

Moderation Bonus = +5

Final Score = 100

Result Published.

Task

1. Preserve the same overall evaluation sequence for every exam type.
2. Allow different exam types to provide their own implementation for individual evaluation steps.
3. Ensure that the final score remains between 0 and 100.
4. Avoid duplicating the complete evaluation workflow separately for each exam type.
5. Make it possible to introduce another exam type in the future with minimal changes to the existing evaluation process.

You may introduce any necessary classes, abstract classes, interfaces, and methods to produce a clean object-oriented solution.